

tf.bitwise.invert Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/bitwise/invert

Invert (flip) each bit of supported types; for example, type uint8 value 01010101 becomes 10101010.

Function Signatures

```
tf.bitwise.invert( x: Annotated[Any, TV_Invert_T], name=None
) -> Annotated[Any, TV_Invert_T]
```

Code Examples

```
tf.bitwise.invert(
x: Annotated[Any, TV_Invert_T], name=None
) -> Annotated[Any, TV_Invert_T]
```

```
tf.bitwise.invert(
x: Annotated[Any, TV_Invert_T], name=None
) -> Annotated[Any, TV_Invert_T]
```

```

import tensorflow as tf
from tensorflow.python.ops import bitwise_ops
# flip 2 (00000010) to -3 (11111101)
tf.assert_equal(-3, bitwise_ops.invert(2))
dtype_list = [dtypes.int8, dtypes.int16, dtypes.int32,
dtypes.int64,
dtypes.uint8, dtypes.uint16, dtypes.uint32, dtypes.uint64]
inputs = [0, 5, 3, 14]
for dtype in dtype_list:
# Because of issues with negative numbers, let's test this
indirectly.
# 1. invert(a) and a = 0
# 2. invert(a) or a = invert(0)
input_tensor = tf.constant([0, 5, 3, 14], dtype=dtype)
not_a_and_a, not_a_or_a, not_0 = [bitwise_ops.bitwise_and(
input_tensor, bitwise_ops.invert(input_tensor)),
bitwise_ops.bitwise_or(
input_tensor, bitwise_ops.invert(input_tensor)),
bitwise_ops.invert(
tf.constant(0, dtype=dtype))]
expected = tf.constant([0, 0, 0, 0], dtype=tf.float32)
tf.assert_equal(tf.cast(not_a_and_a, tf.float32), expected)
expected = tf.cast([not_0] * 4, tf.float32)
tf.assert_equal(tf.cast(not_a_or_a, tf.float32), expected)
# For unsigned dtypes let's also check the result directly.
if dtype.is_unsigned:
inverted = bitwise_ops.invert(input_tensor)
expected = tf.constant([dtype.max - x for x in inputs],
dtype=tf.float32)
tf.assert_equal(tf.cast(inverted, tf.float32), tf.cast(expected,
tf.float32))

```

```

import tensorflow as tf
from tensorflow.python.ops import bitwise_ops
# flip 2 (00000010) to -3 (11111101)
tf.assert_equal(-3, bitwise_ops.invert(2))
dtype_list = [dtypes.int8, dtypes.int16, dtypes.int32,
dtypes.int64,
dtypes.uint8, dtypes.uint16, dtypes.uint32, dtypes.uint64]
inputs = [0, 5, 3, 14]
for dtype in dtype_list:
# Because of issues with negative numbers, let's test this
indirectly.
# 1. invert(a) and a = 0
# 2. invert(a) or a = invert(0)
input_tensor = tf.constant([0, 5, 3, 14], dtype=dtype)
not_a_and_a, not_a_or_a, not_0 = [bitwise_ops.bitwise_and(
input_tensor, bitwise_ops.invert(input_tensor)),
bitwise_ops.bitwise_or(
input_tensor, bitwise_ops.invert(input_tensor)),
bitwise_ops.invert(
tf.constant(0, dtype=dtype))]
expected = tf.constant([0, 0, 0, 0], dtype=tf.float32)
tf.assert_equal(tf.cast(not_a_and_a, tf.float32), expected)
expected = tf.cast([not_0] * 4, tf.float32)
tf.assert_equal(tf.cast(not_a_or_a, tf.float32), expected)
# For unsigned dtypes let's also check the result directly.
if dtype.is_unsigned:
inverted = bitwise_ops.invert(input_tensor)
expected = tf.constant([dtype.max - x for x in inputs],
dtype=tf.float32)
tf.assert_equal(tf.cast(inverted, tf.float32), tf.cast(expected,
tf.float32))

```

Documentation

Invert (flip) each bit of supported types; for example, type uint8 value 01010101 becomes 10101010.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.bitwise.invert`

Flip each bit of supported types. For example, type int8 (decimal 2) binary 00000010 becomes (decimal -3) binary 11111101.

This operation is performed on each element of the tensor argument `x`.

Example:

`## Returns`